Lecture 20 : Project Estimation Techniques (Contd.)

Introduction

Accurately estimating the size and effort required for a software project is a cornerstone of successful project management. Without reliable estimates, projects can suffer from budget overruns, missed deadlines, and scope creep, leading to dissatisfied stakeholders. This lecture builds upon previous discussions of parametric models for size estimation, delving deeper into various function point methods that offer a more abstract and language-independent approach compared to Lines of Code (LOC).

We will revisit the Albrecht/IFPUG function point method with additional practical examples to solidify understanding. Furthermore, we will explore alternative function point techniques, including Symons Mark II and COSMIC function points, which address specific limitations and application domains, such as embedded systems. Understanding these diverse estimation techniques is crucial for software engineers and project managers to select the most appropriate method for their unique project contexts, ensuring more predictable and manageable development cycles.

Core Concepts

Term	Definition	Significance
`Unadjusted Function Point (UFP)`	A preliminary count of functional units (inputs, outputs, inquiries, files, interfaces) in a software system, weighted by their complexity (simple, average, complex).	Provides a raw measure of functional size before considering environmental and technical factors, forming the basis for further refinement.
`Value Adjustment Factor (VAF)`	A multiplier derived from the Total Degree of Influence (TDI) of 14 general system characteristics, reflecting the project's overall complexity and environment.	Refines the UFP to account for non-functional requirements and project characteristics, leading to a more realistic size estimate.

Term	Definition	Significance
`Adjusted Function Point (AFP)`	The final function point count, calculated by multiplying the UFP by the VAF.	Represents the most accurate functional size estimate for a project, incorporating both functional and non-functional aspects.
`Symons Mark II Function Point`	An alternative function point method that estimates size based on the count of data items input, data items output, and entity types accessed per transaction.	Offers a simpler and more usable method, particularly popular in the UK, by focusing on transaction-level data flow.
`COSMIC Full Function Point (FFP)`	A function point method specifically designed for embedded and real-time systems, extending the concept to handle layered architectures and data movements.	Addresses the limitations of traditional FPs for embedded systems by focusing on data movements (entries, exits, reads, writes) within a layered software component.
`Feature Point Metric`	An extension of the function point metric that incorporates an additional parameter: algorithmic complexity.	Overcomes a major drawback of traditional FPs by acknowledging that the effort for a function depends on its algorithmic complexity, not just its type.
`Object Points`	An estimation technique that quantifies project size based on the number of screens, reports, and modules in the code.	Provides a simpler estimation method, particularly useful for projects with significant Graphical User Interface (GUI) components, and is distinct from object-oriented programming.
`Total Degree of Influence (TDI)`	The sum of influence ratings (0-5) for 14 general system characteristics, used to	Quantifies the impact of various technical and environmental factors on

Term	Definition	Significance
	calculate the Value Adjustment Factor (VAF).	project effort, directly influencing the VAF.

Detailed Analysis

1. Albrecht/IFPUG Function Point Examples and Calculation

The Albrecht/IFPUG function point method is a widely used technique for estimating software size based on its functional requirements. The process involves two main steps: calculating the Unadjusted Function Point (UFP) and then refining it to the Adjusted Function Point (AFP) using a Value Adjustment Factor (VAF).

- **Calculating Unadjusted Function Points (UFP)**:
- The UFP is determined by categorizing and counting five types of functional units: user inputs, user outputs, user inquiries, internal logical files, and external interface files.
- Each category is assigned a weight based on its complexity (simple, average, or complex).
- The lecture provides examples where all complexity adjustment factors and weighting factors are assumed to be "average" for simplicity.
- **Example 1**:
- Given: 50 user inputs, 40 user outputs, 35 user inquiries, 6 user files, 4 external interfaces.
- Assuming average weights (inputs: 4, outputs: 5, inquiries: 4, files: 10, external interfaces: 7).
- UFP = (50 * 4) + (40 * 5) + (35 * 4) + (6 * 10) + (4 * 7)
- UFP = 200 + 200 + 140 + 60 + 28 = 628
- **Example 2**:
- Given: 32 user inputs, 60 user outputs, 24 user inquiries, 8 files, 2 external interfaces.
- Assuming average weights (inputs: 4, outputs: 5, inquiries: 4, files: 10, external interfaces: 7).
- UFP = (32 * 4) + (60 * 5) + (24 * 4) + (8 * 10) + (2 * 7)
- UFP = 128 + 300 + 96 + 80 + 14 = 618
- **Calculating Value Adjustment Factor (VAF)**:

- The VAF refines the UFP by considering 14 General System Characteristics (GSCs), each rated on a scale of 0 to 5 for its degree of influence.
- The sum of these ratings is the Total Degree of Influence (TDI).
- The formula for VAF is: VAF = 0.65 + (0.01 * TDI).
- In the examples, since all complexity adjustment factors are assumed "average," and the GSCs are also considered "average," each of the 14 GSCs is assigned a rating of 3.
- TDI = 14 * 3 = 42.
- VAF = 0.65 + (0.01 * 42) = 0.65 + 0.42 = 1.07.
- **Calculating Adjusted Function Points (AFP)**:
- The AFP is obtained by multiplying the UFP by the VAF.
- AFP = UFP * VAF.
- **Example 1**: AFP = 628 * 1.07 = 672.
- **Example 2**: AFP = 618 * 1.07 = 661.26.
- **Estimating Project Size in Lines of Code (LOC)**:
- Once the AFP is determined, it can be converted into LOC using a language-specific conversion rate.
- Size (LOC) = AFP * (LOC per FP).
- **Example 1**: Given 70 LOC per FP.
- Size = 672 * 70 = 47040 LOC.
- **Example 2**: If, for instance, 80 LOC per FP were assumed.
- Size = 661.26 * 80 = 52900.8 LOC (approximately).

2. Symons Mark II Function Points

Developed by Charles R. Symons, the Mark II function point method is an alternative to Albrecht's approach, often considered simpler and more usable, particularly in the UK. It was developed in parallel to IFPUG FPs and builds upon Albrecht's foundational work.

- **Core Principle**: For each transaction within a system, the Mark II method counts three parameters:
- Number of data items input (Ni)
- Number of data items output (No)
- Number of entity types accessed (Ne)
- **Calculation Formula**: The function count (FP) for a project is given by:

- FP count = (Ni * 0.58) + (Ne * 1.66) + (No * 0.26)
- The constants (0.58, 1.66, 0.26) are weights derived from experiments conducted by Symons. This method is simpler because it directly applies these weights to the counted items without the extensive adjustment factor calculation seen in IFPUG.

3. COSMIC Full Function Points (FFPs)

Traditional function point methods like IFPUG and Mark II were primarily designed for information systems, which are heavily dominated by input and output operations. They are often unsuitable for embedded and real-time systems, where the software's user might be another hardware device or software component, and features are often "hidden." COSMIC FFPs were developed to address this gap.

- **Addressing Embedded Systems**: COSMIC deals with the complexities of embedded systems by:
- Decomposing the system architecture into a hierarchy of software layers.
- Viewing embedded software as residing in a particular layer, communicating with other layers (above and below) and peer components at the same level.
- The software component to be sized can receive requests from higher layers and make requests to lower layers.
- **Data Movement Types**: COSMIC focuses on four fundamental types of data movement, each counting as one COSMIC functional size unit (Cfsu):
- **Entries**: Movement of data into the software component from a higher layer or a peer component.
- **Exits**: Movement of data out of the software component.
- **Reads**: Data movement from persistent storage.
- **Writes**: Data movement to persistent storage.
- **Calculation**: The overall FFP count is derived by simply adding up the counts for each of these four types of data movement.
- **Standardization**: A significant advantage of COSMIC FFPs is their incorporation into an ISO standard, lending them greater credibility and widespread acceptance compared to some other metrics.
- **Disadvantages**:
- COSMIC FFPs do not account for any processing of data groups once they have moved into the software component.
- They are generally not recommended for systems involving complex mathematical algorithms, as the processing complexity is not considered.

4. Pros and Cons of Function Points

Function point metrics offer several advantages but also come with notable drawbacks.

- **Pros**:
- **Language Independent**: Unlike LOC, FPs are not tied to a specific programming language, making them useful for multi-language projects or early-stage estimation.
- **Understandable by Client**: The focus on functional requirements makes FPs more accessible and understandable to non-technical clients.
- **Simple Modeling**: They provide a relatively straightforward way to model and quantify software functionality.
- **Hard to Fudge**: The structured counting rules make it difficult to artificially inflate or deflate estimates.
- **Visible Feature Creep**: Changes in functional requirements directly impact the FP count, making feature creep more visible.
- **Cons**:
- **Labor Intensive**: Calculating FPs can be time-consuming, especially for large projects.
- **Extensive Training Required**: Accurate FP counting requires significant training and experience.
- **Inexperience May Result in Inaccuracy**: Inexperienced counters can produce inaccurate estimates.
- **Weighted to File Manipulation and Transactions**: Traditional FPs might overemphasize data manipulation and transaction processing, potentially underrepresenting other aspects.
- **Errors by Single Person**: A single estimator might introduce bias or errors; multiple raters are often advised for consistency.
- **Does Not Consider Algorithmic Complexity**: A major limitation is that FPs treat all functions similarly, regardless of their internal algorithmic complexity.

5. Feature Point Metric

To address the significant drawback of function points not considering algorithmic complexity, the Feature Point Metric was proposed as an extension.

- **Core Concept**: The Feature Point Metric introduces an additional parameter: **algorithmic complexity**.

- **Significance**: This parameter ensures that the computed size reflects the fact that higher complexity functions require greater development effort and should therefore have a larger estimated size compared to simpler functions. It acknowledges that a complex algorithm, even if it performs a single "function," demands more effort than a simple data entry function.

6. FP/SLOC Conversion and Accurate Size Estimation

While function points provide a language-independent size estimate, converting them to Lines of Code (SLOC) is often necessary for practical planning, as many development tools and historical data are based on LOC.

- **Language-Specific Conversion**: The conversion rate from function points to SLOC varies significantly across programming languages. For example:
- C: 104 SLOC per function point
- C++: 53 SLOC per function point
- HTML: 42 SLOC per function point
- Java: 59 SLOC per function point
- Perl: 60 SLOC per function point
- J2EE: 50 SLOC per function point
- Visual Basic: 42 SLOC per function point
- **Accurate Size Estimation Model**: To achieve more accurate size estimation, a comprehensive approach involves:
- **Requirement Analysis**: Define project requirements.
- **Estimate Project Size**: Use methods like function point analysis to get an initial size (e.g., UFP).
- **Project Complexity Adjustment Factors**: Apply factors like the VAF from function point analysis.
- **Risk Factors**: Multiply by risk factors to account for uncertainties and potential challenges.
- **Situation Adjustments**: Further adjust based on specific project context.
- This refined size estimate then informs the calculation of effort, cost, and scheduling.

7. Object Points

Object points are another estimation technique, distinct from object-oriented programming, that focuses on user-facing components and internal modules.

- **Estimation Basis**: The number of object points is estimated based on three factors:
- The number of separate screens displayed.
- The number of reports that are produced.
- The number of modules present in the code.
- **Advantages**: Object points are generally simpler to estimate and are particularly effective for projects that involve significant Graphical User Interfaces (GUIs), as they directly account for screens and reports.

Key Takeaways

- [ ] Albrecht/IFPUG function points are calculated by first determining Unadjusted Function Points (UFP) from functional units and then refining them with a Value Adjustment Factor (VAF) to get Adjusted Function Points (AFP).
- [ ] The VAF is derived from the Total Degree of Influence (TDI) of 14 general system characteristics, reflecting non-functional aspects and project environment.
- [ ] Symons Mark II function points offer a simpler estimation method, focusing on data items input, output, and entity types accessed per transaction, with predefined weights.
- [ ] COSMIC Full Function Points (FFPs) are specifically designed for embedded and real-time systems, accounting for data movements (entries, exits, reads, writes) within a layered software architecture.
- [ ] COSMIC FFPs are incorporated into an ISO standard, highlighting their formal recognition and applicability for complex systems.
- [ ] A major limitation of traditional function points is their inability to account for algorithmic complexity, treating all functions as equally complex.
- [ ] The Feature Point Metric extends function points by including algorithmic complexity as an additional parameter, leading to more accurate effort estimation for complex functions.
- [ ] Function points are language-independent and client-understandable, but can be labor-intensive and require experienced estimators.
- [ ] Project size in function points can be converted to Lines of Code (SLOC) using language-specific conversion rates, which vary significantly.

- [ ] Accurate size estimation should incorporate project complexity adjustment factors and risk factors in addition to basic functional size.

Conclusion

This lecture has provided a comprehensive overview of various function point estimation techniques, moving beyond the basic Albrecht/IFPUG method to include Symons Mark II and COSMIC function points. We've seen how each method offers unique advantages and addresses specific project contexts, from traditional information systems to complex embedded environments. The critical discussion around the pros and cons of function points, particularly the limitation regarding algorithmic complexity, highlighted the continuous evolution of estimation models, leading to extensions like the Feature Point Metric.

Understanding these diverse approaches is vital for project managers and software engineers to make informed decisions about project sizing. By carefully considering the nature of the project, the available information, and the desired level of detail, one can select the most appropriate estimation technique. The journey into project estimation continues with the exploration of more advanced models like COCOMO, which will be the focus of our next session, further equipping us with tools for robust software project planning.